

[← GO BACK](#)

Signals: Toward a Self-Improving Agent

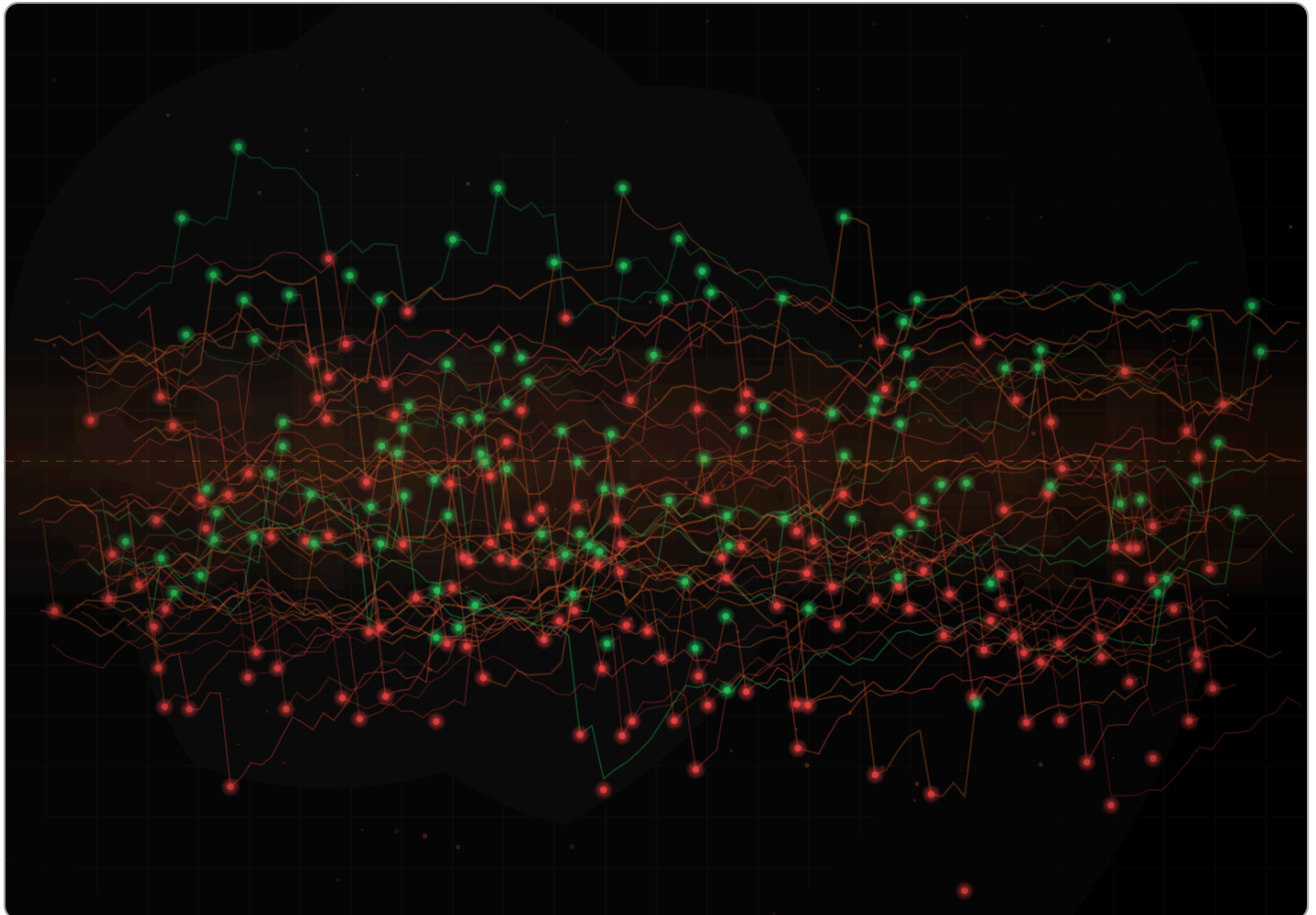
BY FACTORY RESEARCH - JANUARY 23, 2026 - 6 MINUTE READ - SHARE  

[ENGINEERING](#)[RESEARCH](#)[NEW](#)

How we built a closed-loop system for recursive self-improvement—where the agent detects its own failures and implements fixes automatically.



01 THE PROBLEM WITH METRICS



Traditional product analytics tell you what happened. Session duration, tool calls executed, completion rates. But they rarely tell you how it felt. A user might complete a task successfully in twenty minutes, but did they spend eighteen of those minutes fighting the tool?

We built Signals to answer that question. Signals uses LLMs as judges to analyze Factory sessions at scale, identifying moments of friction and delight that metrics alone would miss. More importantly, it does this without anyone ever reading user conversations. And when friction crosses a threshold, Droid fixes itself. This is recursive self-improvement: the agent analyzing its own behavior and evolving autonomously.



Toggle between aggregate view (average sentiment with confidence band) and individual sessions. Click a session line to see friction and delight moments. Hover over points to reveal citations.

The Problem with Metrics

Consider a session where a developer asks Droid to refactor a module. The metrics look fine: forty-five tool calls, twelve-minute session, task completed. But buried in that session is a three-minute loop where the user rephrased the same request five times, growing increasingly frustrated, before the agent finally understood what they wanted.

Traditional analytics would score this session as a success. A human reviewer would call it a near-disaster. We needed a system that could think more like the human, without actually requiring humans to read through thousands of daily sessions.

How Signals Works

Signals processes sessions using LLM and embedding-based analysis. The model never surfaces raw conversation content to human analysts. Instead, it extracts abstract patterns and categorized signals that tell us what happened without revealing what was said.

Facet Extraction

Every session gets decomposed into structured metadata. We call these facets: the programming languages involved, the primary intent, how many tool calls were confirmed, whether the session ended in success or abandonment, what frameworks were referenced. These facets enable aggregate analysis across thousands of sessions without anyone reading the underlying conversations.

The facet schema itself evolves over time through semantic clustering. As Signals processes batches of sessions, it generates embeddings for each session's abstracted summary and clusters similar sessions together. The LLM then analyzes these clusters to identify new facet categories worth tracking. When a cluster emerges that doesn't map cleanly to existing facets, Signals proposes a new dimension.

Early versions of Signals had no concept of "branch switches" as a facet. The clustering revealed a group of sessions that shared similar patterns but didn't fit existing categories. When the LLM examined what these sessions had in common, it identified that git branch changes correlated with session complexity and surfaced it as a new dimension worth tracking.

Friction Detection

Friction Signal Types				
Seven example indicators*				
Error Events	Model errors, tool failures, timeouts	35%	45%	20%
Repeated Rephrasing	≥3 consecutive restating messages	42%	38%	20%

Escalation Tone	"broken", "why isn't", "frustrating"	28%	52%	20%
Platform Confusion	Questions about Factory features	15%	55%	30%
Abandoned Tool Flow	Tool calls rejected or cancelled	48%	32%	20%
Backtracking	"undo", "revert", deleting code	22%	48%	30%
Context Churn	Add/remove same file repeatedly	38%	42%	20%

High Severity

Medium

Low

The friction analyzer scans for patterns that indicate user struggle: error events, repeated rephrasing, escalation in tone, tool calls rejected by the user, and more. Each friction moment gets a severity rating and abstracted citations that describe what happened without exposing user quotes, code, or PII. A citation might read "user expressed frustration after third failed tool call" rather than quoting the user's actual words.

Like facets, friction categories evolve through the same embedding and clustering process. Signals generates embeddings for friction descriptions and clusters them to find recurring patterns that don't fit existing categories. When a new cluster emerges across enough sessions, it proposes a new friction type.

The "context churn" category didn't exist in our original design. Signals' clustering surfaced a group of friction moments that shared semantic similarity but didn't match any existing type. When the LLM examined the cluster, it identified the common thread: users repeatedly modifying their context window in ways that correlated with eventual abandonment. That pattern became a first-class friction category.

Delight Identification

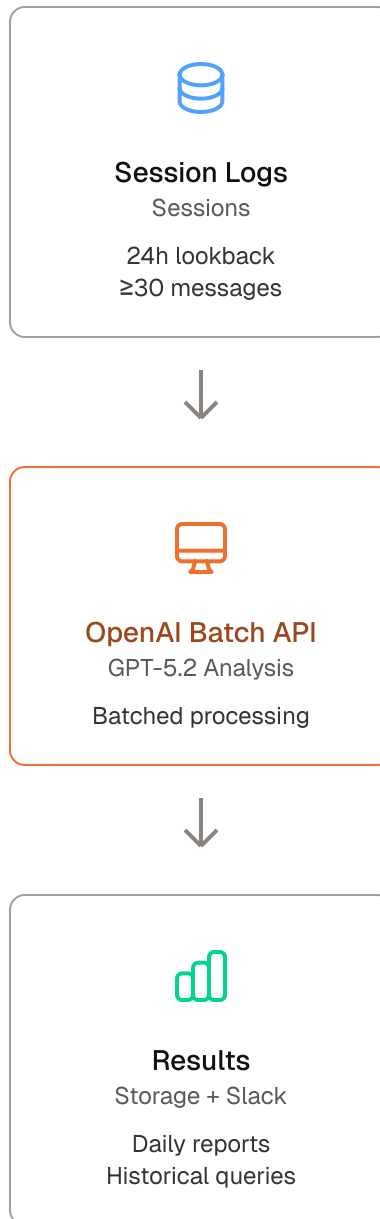
Signals doesn't just find problems. It finds moments where Factory genuinely impressed users. Positive exclamations, first-attempt successes on complex tasks, explicit mentions of time saved, rapid approval flows followed by appreciation. These moments matter for understanding what to do more of, not just what to fix.

Delight categories evolve through the same mechanism. The system recently surfaced "learning moments" as a new delight type after discovering that sessions where Droid explained its reasoning generated disproportionately positive signals compared to sessions that just executed without explanation.

The Pipeline

Signal Pipeline Architecture

Analyzes thousands of sessions per day



Signals runs as a daily batch process designed for scale and cost efficiency. Sessions from the past twenty-four hours get fetched from BigQuery, filtered to those with at least thirty agentic steps to ensure meaningful interactions, and sent to OpenAI's batch API.

We analyze thousands of sessions daily, dynamically adjusted based on a token budget. Batching lets us take advantage of lower API costs, and the twenty-four hour processing window works well since we're looking for

patterns, not real-time alerts.

Results flow to BigQuery for historical analysis and to Slack for daily reports. The BigQuery tables let us query friction patterns over time, correlating them with releases and feature changes. The Slack reports give the team a daily pulse on how users experienced the product.

Correlating with System Behavior

Signals becomes even more powerful when correlated with our internal logging and release data.

We pipe error logs from our observability system into Signals' analysis. When a session shows friction, Signals can cross-reference with backend errors that occurred during the same time window. This surfaces patterns like "users experience repeated rephrasing friction when the context assembly service throws timeout errors" without anyone manually investigating individual sessions.



#evaluation • Factory Signal Bot

S

Signal Bot 3:00 PM



Daily Friction Analysis Report (2026-01-16)

Generated at 2026-01-16 15:00:00



Summary:

- Sessions analyzed: **1,946** (39 batches)
- Sessions with friction: **34** (27%)
- Total friction points: **89**
- Average per affected session: **2.6** friction points
- Severity breakdown: **12 High**, **41 Medium**, **36 Low**



Top Friction Sessions:

1. [session_a7f2e1](#) - 7 friction points (3 high)
 - User experienced repeated tool failures during git operations
2. [session_b3c9d4](#) - 5 friction points (2 high)
 - Multiple rephrasing attempts for code generation task
3. [session_e8f1a2](#) - 4 friction points (1 high)
 - Platform confusion around context management

Daily analysis aggregated from 3 batch(es)

Daily Slack reports now include correlation with our release notes. When a new CLI version ships, Signals automatically tracks whether friction patterns change in subsequent sessions. We've caught regressions this

way. A release that changed how file context was assembled correlated with a spike in context churn friction the following day. The aggregate pattern was visible immediately; investigating would have taken weeks of manual review.

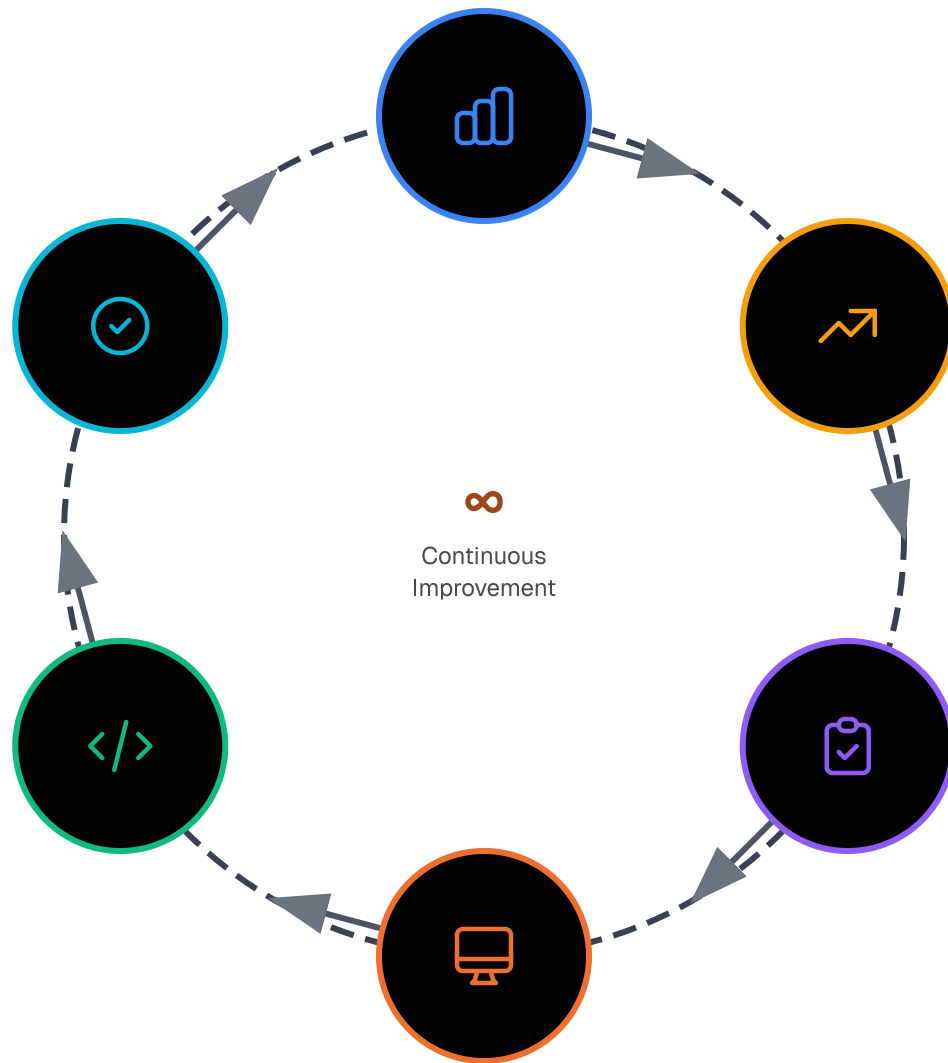
We can also track improvements. When we shipped a change to how Droid handles ambiguous requests, the "repeated rephrasing" friction rate dropped by thirty percent within forty-eight hours. Signals surfaced this without anyone having to read before-and-after sessions to verify the fix worked.

Closing the Loop

Signals implements what AI researchers call recursive self-improvement—systems that autonomously enhance their own capabilities. When patterns cross a threshold, it files Linear tickets automatically. Droid picks up those tickets, implements fixes, and reviews its own PRs.

The Self-Improving Loop

When friction crosses thresholds, Droid files tickets, assigns itself, and implements fixes



1

Signal Detects
Friction patterns

3

Ticket Filed
Linear issue created

5

Fix Implemented
PR created

2

Threshold Crossed
Pattern frequency

4

Droid Assigned
Self-assignment

6

Droid Review
Human Approved

73%

Issues auto-resolved

<4h

Avg time to fix

1

Human approval step

It's not fully automated yet. A human still approves the PR before merge. But the path from "users are frustrated by X" to "here's a fix for X" now happens without anyone manually triaging, assigning, or even noticing the pattern. Recent examples include tickets for improving tool timeout handling after Signals detected it was responsible for over half of high-severity friction, and fixing output truncation that was delivering incomplete code to users. We're adding more automation to this loop every month.

Privacy Without Blindness

Traditional approaches force a choice: either read user sessions to understand problems, or stay blind to preserve privacy. Signals resolves this through multiple layers of abstraction. The LLM extracts patterns while omitting specific user content. Individual results flow into aggregate statistics that only become meaningful at scale. And patterns only surface when they appear across enough distinct sessions to prevent identifying individual users. We know which friction patterns correlate with abandonment without anyone at Factory reading a single user conversation.

What We've Learned

After running Signals for several months, patterns have emerged that we couldn't have found through traditional metrics.

Context churn turned out to be the leading indicator of eventual frustration. When users repeatedly add and remove the same file from context, something fundamental is wrong. Either the file isn't being read correctly, or the agent isn't using it as expected. This pattern often appears minutes before more obvious friction signals like escalation in tone.

Rephrasing cascades predict abandonment with surprising accuracy. If a user rephrases three times, there's roughly a forty percent chance they'll rephrase again. If they hit five rephrases, session completion rates drop significantly. This insight led us to implement proactive clarification when Droid detects potential ambiguity, rather than waiting for the user to rephrase.

Error recovery matters more than error prevention. This one surprised us. Sessions that hit errors but recovered gracefully actually scored higher on delight than sessions with no errors at all. Users seem to appreciate resilience over perfection. A system that fails and recovers builds more trust than one that works flawlessly but feels fragile.

On the delight side, the most common positive signal is efficiency. The phrase "would have taken me hours" appears in abstracted form across hundreds of delight citations. Users don't expect the agent to be faster than manual work. When it is, they notice.

What's Next

Signals is the foundation for something more ambitious: a self-evolving agent capable of true recursive self-improvement. Today, the loop runs daily. Tomorrow, it runs in real-time, surfacing friction indicators during active sessions so the agent can course-correct before frustration builds.

Beyond reactive fixes, we're building toward proactive evolution. Signals doesn't just identify what's broken, it identifies what's missing. When clusters reveal users repeatedly asking for capabilities that don't exist, that's signal for what to build next. The evolution mechanism continues to find new patterns—last week it proposed tracking "specification drift," sessions where the user's stated goal shifted mid-conversation.

The end state is an agent that learns from every interaction, improves continuously, and evolves its own capabilities over time. Signals is how we get there.

START BUILDING



Ready to build the software of the future?

START BUILDING →

Resources	Company	Legal
News	Careers	Privacy Policy
Docs	Enterprise	Terms of Service
Contact Sales	Security	SLA
Open Source		DPA
		BAA

🌙 ☀️ 🗨 SYSTEM



X (Twitter), LinkedIn, GitHub
@Factory 2026. All rights reserved.